

AIM:

[illegible]

|||||;

```
//<<<<<<<<<<<<<<<<<<<<<<<<Code Begins Here>>>>>>>>>>>>>>>>>
```

```

void Delay()
{
    unsigned int i,j;
    for(i=0;i<1000;i++)
        for(j=0;j<700;j++);
}

void main()
{
    PINSEL0    =    0x00;        //Configure Port0.7 as GPIO
    IODIR0     =    3 << BUZZ;   //Configure Port0.7 as OutPut pin
    while(1)
    {
        IOSET0 = 1 << BUZZ;
        Delay();
        IOCLR0 = 1 << BUZZ;
        Delay();

    }
}

```

RESULT:

REG NO: 22TD0005 NAME: AKSHAYA S YEAR/SEM/SEC: Y3/S6/B

AIM:

SOURCE CODE:

```
;//////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////;
;//////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////;

;|      Title      : LCD Demo.c                                     |;
;|      Status     : Completed                                     |;
;|      Features    : Program to Display Message in 2 lines      |;
;|      Description : LCD Used :2x16 LCD in 4-Bit Mode           |;
;|      Control Lines :P0.16 (RS) | P0.17 (RW) | P0.18 (EN)      |;
;|      Data Lines   :P0.19 - P0.22                               |;
;|      Enable LCD, turn on switch SW28 at LCD label mark position |;
;//////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////;

#include <LPC214x.H>

#define RS      0x10000

#define RW      0x20000

#define EN      0x40000

void lcd_cmd (unsigned char);

void lcd_data (unsigned char);

void lcd_initialize (void);

void lcd_display (void);

void LCD4_Convert(unsigned char);

const unsigned char cmd[4] = {0x28,0x0c,0x06,0x01};           //lcd commands

unsigned char msg[] = {">PS-Primer 2148<"};                  //msg
```

```

unsigned char msg1[]= {":: LCD Demo! ::"};                                //msg1

void delay(unsigned int n)
{
    unsigned int i,j;
    for(i=0;i<n;i++)
    {
        for(j=0;j<12000;j++)
        {;}
    }
}

//-----
//          LCD Command Send
//-----

void lcd_cmd(unsigned char data)
{
    IOCLR0    |= RS;                //0x1000;          //RS
    IOCLR0    |= RW;                //0x2000;          //RW
    LCD4_Convert(data);
}

void lcd_initialize(void)
{
    int i;
    for(i=0;i<4;i++)
    {
        IOCLR0 = 0xF << 19;        //IOCLR 0/1
        lcd_cmd(cmd[i]);
    }
}

```

```

        delay(15);

    }

}

//-----
//          LCD Data Send
//-----

void lcd_data (unsigned char data)
{
    IOSET0      |= RS;          //0x1000;      //RS
    IOCLR0      |= RW;          //0x2000;      //RW
    LCD4_Convert(data);
}

//-----
//          LCD Display Msg
//-----

void lcd_display (void)
{
    char i;

    /* first line message */

    lcd_cmd(0x80);

    delay(15);

    i=0;

    while(msg[i]!='\0')
    {

        delay(5);

        lcd_data(msg[i]);
    }
}

```

```

        i++;

    }

    delay(15);

    /* second line message */

    lcd_cmd(0xc0);

    delay(15);

    i=0;

    while(msg1[i]!='\0')
    {

        delay(5);

        lcd_data(msg1[i]);

        i++;

    }

    delay(15);
}

void LCD4_Convert(unsigned char c)
{

    if(c & 0x80) IOSET0 = 1 << 22; else IOCLR0 = 1 << 22;

    if(c & 0x40) IOSET0 = 1 << 21; else IOCLR0 = 1 << 21;

    if(c & 0x20) IOSET0 = 1 << 20; else IOCLR0 = 1 << 20;

    if(c & 0x10) IOSET0 = 1 << 19; else IOCLR0 = 1 << 19;


    IOSET0      = EN;
                //0x4000;                //EN

    delay(15);

    IOCLR0      = EN;
                //0x4000;                //EN

```

```

        if(c & 0x08) IOSET0 = 1 << 22; else IOCLR0 = 1 << 22;

        if(c & 0x04) IOSET0 = 1 << 21; else IOCLR0 = 1 << 21;

        if(c & 0x02) IOSET0 = 1 << 20; else IOCLR0 = 1 << 20;

        if(c & 0x01) IOSET0 = 1 << 19; else IOCLR0 = 1 << 19;

        IOSET0      = EN;
                //0x4000;                //EN

        delay(15);

        IOCLR0      = EN;
                //0x4000;                //EN
    }

    void main()

    {

        PINSEL1      =      0;
        //Configure P0.16 - P0.31 as GPIO

        IODIR0       =      0xFF << 16;    //Configure Pins P0.16 - P0.22 as Output Pins

        lcd_initialize();                    //Initialize LCD!

        lcd_display();                      //Display Message in LCD

        while(1);

    }

```

RESULT:

AIM:

```
#define Switch 24
```

```
/****** Declarations *****/
```

```
void Delay(int);
```

```
/******  
/
```

```
int main(void)
```

```
{
```

```
    unsigned char Status=1;
```

```
    PINSEL2    &=    0xFFFFFFF3;    //Configure P1.16 - P1.31 as GPIO
```

```
    IO1DIR     =     0x00 << Switch;    //Configure P1.24 - P1.31 as Input
```

```
    IO1DIR     |=     0xFF << LED;    //Configure P1.16 - P1.23 as Output
```

```
    IO1PIN=     0;
```

```
    while(1)
```

```
    {
```

```
        Status = 1;
```

```
        IOSET1     =     0xFF << LED;
```

```
        Delay (10);Delay (10);
```

```
        IOCLR1     =     0xFF << LED;
```

```
        Delay (10);Delay (10);Delay (10);
```

```
        while (~Status)
```

```
        {
```

```
            Status = ((IO1PIN & (0xFF << Switch)) >> Switch);
```

```
            IO1PIN= ((~Status) << LED);
```

```
        }  
    }  
}
```

```
void Delay(int n)  
{  
    int p,q;  
  
    for(p=0;p<n;p++)  
    {  
        for(q=0;q<0x9990;q++);  
    }  
}
```

RESULT:

REG NO: 22TD0005 NAME: AKSHAYA S YEAR/SEM/SEC: Y3/S6/B

AIM:

SOURCE CODE:

```
#include <lpc214x.h>                //Includes LPC2148 register definitions

#define SW3          1<<24

#define SW4          1<<25

#define SW5          1<<26


#define MAX_BUFFER_SIZE 16

#define DEVICE_ADDR 0xA0

#define BLK_0 0x00

#define BLK_1 0x02

#define MAX_BLOCK_SIZE 256


#define LED1_ON() IO1SET=(1<<16) //I2C write indicator

#define LED2_ON() IO1SET=(1<<17) //I2C read indicator

#define LED3_ON() IO1SET=(1<<18) //Comm failure indicator

#define LED4_ON() IO1SET=(1<<19) //Comm success indicator


#define LED1_OFF() IO1CLR=(1<<16)

#define LED2_OFF() IO1CLR=(1<<17)

#define LED3_OFF() IO1CLR=(1<<18)

#define LED4_OFF() IO1CLR=(1<<19)
```

```

#define Fosc      12000000

#define Fcclk     (Fosc * 5)

#define Fcco      (Fcclk * 4)

#define Fpclk     (Fcclk / 4)


void Delay(unsigned char j);

void Send_Start(void);

void Send_Stop(void);

unsigned char Send_I2C(unsigned char *Data,unsigned char Len);

unsigned char Read_I2C(unsigned char *Data,unsigned char Len);

unsigned char Page_Write(unsigned char BLOCK_NUMBER,unsigned char
BLOCK_ADDR,unsigned char *str);

unsigned char Page_Read(unsigned int BLOCK_NUMBER,unsigned char BLOCK_ADDR);

unsigned char I2C_Status(unsigned char status_code);

unsigned char I2C_WR_Buf[MAX_BUFFER_SIZE]="ARM7 PRIMER BRD ";

unsigned char I2C_RD_Buf[MAX_BUFFER_SIZE];

unsigned char I2C_ER_Buf[MAX_BUFFER_SIZE]={0,0,0,0,0,0,0,0,0,0,0,0,0,0};

unsigned char Status=0;

unsigned char Status_Flag=0;


void Delay(unsigned char j)
{
    unsigned int i;
    for(;j>0;j--)
    {

```

```
    for(i=0; i<60000; i++);  
}  
}
```

```
void Send_Start()  
{  
    I2C1CONSET=0x20;  
}
```

```
void Send_Stop()  
{  
    I2C1CONSET=0x10;  
}
```

```
// This function sends sequential data to the EEPROM 24LC04  
// The buffer size for EEPROM 24LC04 is 16 bytes  
// The Len parameter should not exceed this value  
unsigned char Send_I2C(unsigned char *Data,unsigned char Len)  
{  
    while(Len)  
    {  
        I2C1DAT=*Data;  
        if(I2C_Status(0x28))  
        {  
            return 1;  
        }  
    }
```

```

    Len--;

    Data++;
}

return 0;
}

```

// This function reads random data from the EEPROM 24LC04

```

unsigned char Read_I2C(unsigned char *Data,unsigned char Len)

```

```

{
    while(Len)
    {
        if(Len==1) //Last byte
        {
            I2C1CONCLR=0x04; //set hardware to send nack

            if(I2C_Status(0x58)) //last byte has been received and NACK has been returned
            {
                return 1;
            }

            *Data=I2C1DAT;
        }
        else
        {
            I2C1CONSET=0x04; //set hardware to send ack

            if(I2C_Status(0x50)) //Byte has been received ACK has been returned
            {

```

```

    return 1;

}

*Data=I2C1DAT;

}

Data++;

Len--;

}

return 0;

}

```

```

unsigned char I2C_Status(unsigned char status_code)

{

while(Status_Flag==0);

Status_Flag=0;

if(Status!=status_code)

{

return 1;

}

else

{

return 0;

}

}

```

```

unsigned char Page_Write(unsigned char BLOCK_NUMBER,unsigned char BLOCK_ADDR,
unsigned char *str)

```

```

{
    Send_Start();

    if(I2C_Status(0x08))    //Start has been transmitted
    {
        return 1;
    }

    I2C1DAT=DEVICE_ADDR | BLOCK_NUMBER;    // Send Address

    if(I2C_Status(0x18))    //Device address, block num and write has
    been transmitted
    {
        return 1;
    }

    I2C1DAT=BLOCK_ADDR;    // Send block address

    if(I2C_Status(0x28))    //Block address has been transmitted
    {
        return 1;
    }

    if(Send_I2C(str,MAX_BUFFER_SIZE))    //Send Data
    {
        Send_Stop();

        return 1;
    }

    Send_Stop();

    return 0;
}

```

```
}
```

```
unsigned char Page_Read(unsigned int BLOCK_NUMBER,unsigned char BLOCK_ADDR)
```

```
{
```

```
Send_Start();
```

```
if(I2C_Status(0x08)) //Start has been transmitted
```

```
{
```

```
return 1;
```

```
}
```

```
I2C1DAT=DEVICE_ADDR | BLOCK_NUMBER; // Send Address
```

```
if(I2C_Status(0x18)) //Device address, block num and write has been transmitted
```

```
{
```

```
return 1;
```

```
}
```

```
I2C1DAT=BLOCK_ADDR;
```

```
if(I2C_Status(0x28)) //Block address has been transmitted
```

```
{
```

```
return 1;
```

```
}
```

```
Send_Start(); // Repeat Start
```

```
if(I2C_Status(0x10)) //Repeated Start has been transmitted
```

```
{
```

```
return 1;
```

```
}
```

```
I2C1DAT=DEVICE_ADDR | BLOCK_NUMBER | 0x01; //Device address, block num and read  
has been transmitted
```

```
if(I2C_Status(0x40)) //
```

```
{
```

```
    return 1;
```

```
}
```

```
if(Read_I2C(I2C_RD_Buf,MAX_BUFFER_SIZE)) //Receive 16bytes of Data from EEPROM
```

```
{
```

```
    Send_Stop();
```

```
    return 1;
```

```
}
```

```
Send_Stop();
```

```
return 0;
```

```
}
```

```
void __irq I2C1_Status(void)
```

```
{
```

```
    Status_Flag=0xFF; //update status flag
```

```
    Status=I2C1STAT; //Read Status byte
```

```
    I2C1CONCLR=0x28;
```

```
    VICVectAddr = 0x00; //Acknowledge Interrupt
```

```
}
```

```
void I2C_Init()
```

```

{
    PINSEL0|=0x30C00000;

    I2C1CONCLR=0x6C;

    I2C1CONSET=0x40;

    I2C1SCLH =80;

    I2C1SCLL =70;


    /* Init VIC for I2C1 */

    VICIntSelect = 0x00000000;    // Setting all interrupts as IRQ(Vectored)

    VICVectCntl0 = 0x20 | 19;    // Assigning Highest Priority Slot to I2C1 and enabling this slot

    VICVectAddr0 = (unsigned long)I2C1_Status; // Storing vector address of I2C1

    VICIntEnable = (1<<19);


}


void UART0_Init(unsigned int baudrate)
{
    unsigned int BAUD_VAL;

    BAUD_VAL = (Fpclk/(16*baudrate));


    PINSEL0 |= 0x00000005;    // Enable rx,tx

    U0LCR = 0x00000083;    // 8 bit data,1 stop bit,no parity bit

    U0DLL = BAUD_VAL & 0xFF; // U0DL for low byte

    U0DLM = (BAUD_VAL>>8); // U0DL for high byte

    U0LCR = 0x00000003;    // DLAB =0

}

```

```

void UART0_Txmt(unsigned char Chr) /* Write character to Serial Port */
{
    while (!(U0LSR & 0x20));
    U0THR = Chr;
}

```

```

unsigned char UART0_Recv(void) /* Write character to Serial Port */
{
    while (!(U0LSR & 0x01));
    return (U0RBR);
}

```

```

void UART0_puts(unsigned char *string)
{
    while(*string)
        UART0_Txmt(*string++);
}

```

```

int main(void)
{

```

```

    PINSEL2 = 0;

```

```

    IO1DIR = (1<<19) | (1<<18) | (1<<17) | (1<<16); // Set P1.16, P1.17, P1.18, P1.19 as Output

```

```

    IO1DIR &= 0xfC7ffff;

```

```
while (!(IOPIN1 & SW3));
```

```

}

else if ((IOPIN1 & SW4) == 0) /*..To Read the Data Stored in the EEPROM...*/
{
    LED2_ON();    //Read indicator

    if(Page_Read(BLK_1,0x00))
    {
        UART0_puts("Read Failed");

        LED4_ON();
    }
    else
    {
        UART0_puts("Data Read:");

        UART0_puts(I2C_RD_Buf);

        UART0_puts("\r\n");
    }

    Delay(50);

    LED2_OFF();

    while (!(IOPIN1 & SW4));
}

else if ((IOPIN1 & SW5) == 0)
{
    LED3_ON();    //Write Indicator

    if(Page_Write(BLK_1,0x00,I2C_ER_Buf))
    {
        UART0_puts("Write Failed");

        LED4_ON();
    }
}

```

```
    }  
    Delay(50);  
    LED3_OFF();  
    UART0_puts("Data Erase Successfully\r\n");  
    while (!(IOPIN1 & SW5));  
}  
  
}  
  
}
```

RESULT: